Autobelay

long premium, short leash

An autonomous options agent on Alpaca's MCP server.
Nobody holds the other end. The brake is code.

Team RazorsEdge - William P. McCormick / William C. McCormick
account `PA3VS39Y5LE2` - MIT - Alpaca AI Trading Agents Hackathon,
Aug 28 to Sep 4, 2026

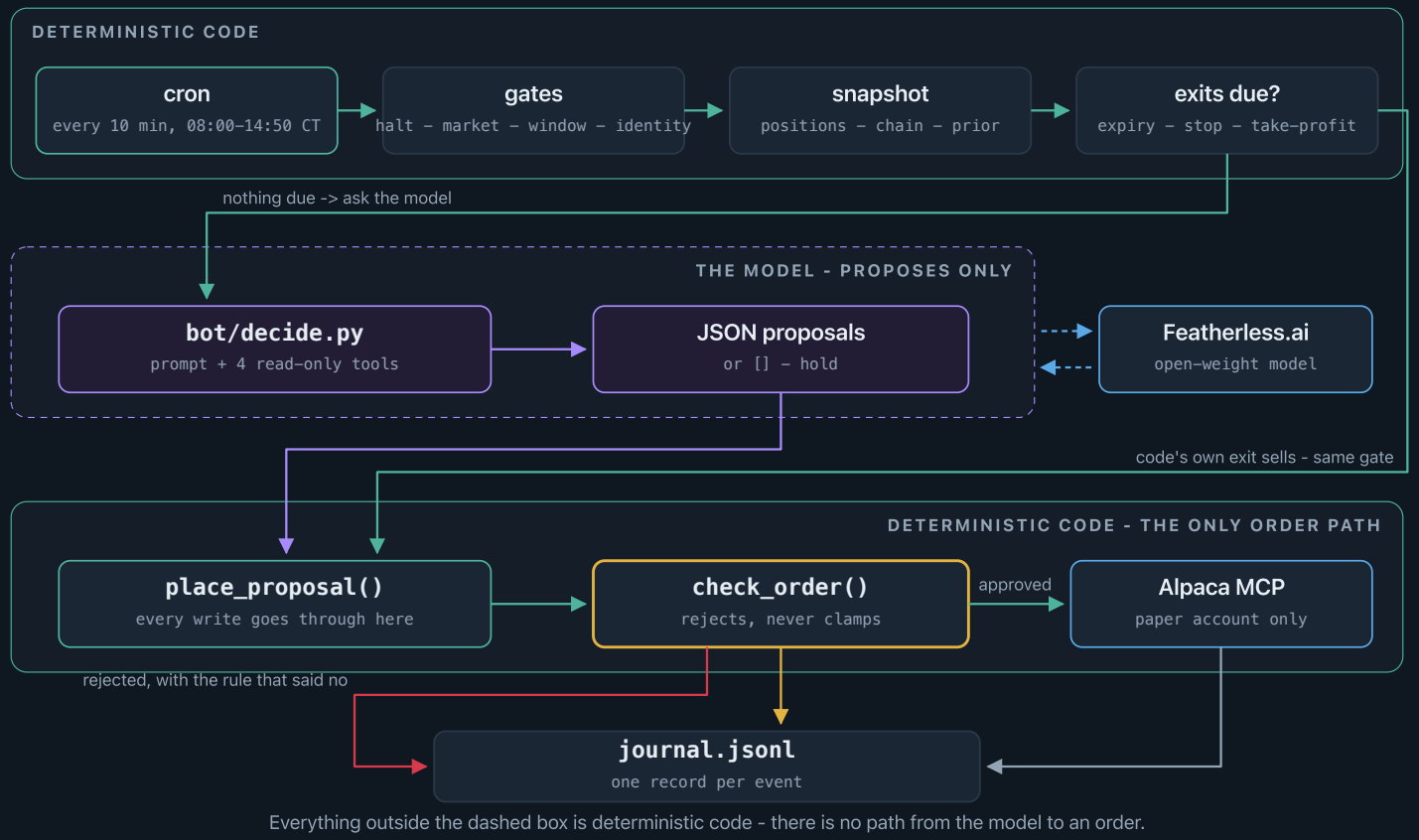
The thesis, in one sentence

Buy defined-risk, short-dated options premium on the most liquid names when an open-source model can *name a reason*; deterministic code sizes every trade, stops it, and closes it before expiry.

The model never touches an order. Open model proposes; code disposes.

Why long premium: the worst case is known before entry, which keeps every guardrail simple and absolute - and it matches what a language model is good at (a directional view from evidence) rather than what it is bad at (managing a position minute to minute).

One cycle, every 10 minutes



Everything outside the dashed box is deterministic code - there is no path from the model to an order.

The model occupies one dashed box and emits JSON. Every order - its proposals *and* code's own exit sells - goes through the same `check_order()`.

Greeks: Alpaca's, with Black-Scholes as the backstop

We spent three days believing the free feed carried no Greeks, and solved implied volatility from each contract's price ourselves. **The belief was wrong.** Alpaca's option snapshot carries IV and the full Greeks on most contracts - today, **5,721 of 6,114** in the fetched chain.


```
{"symbol":"SPY260903P00760000","strike":760.0,"dte":2,"bid":2.20,"ask":2.21,
 "spread_pct":0.5,"iv":0.1368,"delta":-0.3992,"greeks_source":"alpaca"}
```


So the agent uses Alpaca's, and `bot/greeks.py` - a bisection IV solve, robust far from the money - is now the *backstop* for the ~6% too thin to price. Every contract carries `greeks_source`, so the model knows which numbers are rough.

Worth having both: one vendor surface means a strike's put and call deltas agree, where our own solve drifted about five delta points on NVDA. And a feed that goes quiet does not take the whole menu with it.

What the model is allowed to see

One `get_option_chain` page is 500 contracts. On SPY, at $1 strikes with daily expiries, 500 contracts is **three days** - so a config that said 2-45 DTE was quietly showing the model a three-day window, and it reached off-menu for contracts it could not price.

	before	now
SPY chain fetched	500 contracts, 1 page, out to 3 DTE	2,912 contracts, 3 pages, out to 45 DTE
NVDA's 12-contract menu	all at one strike (K=220), six expiries	at-the-money *and* ~0.40 delta, three expiries, both sides

Coverage is journaled every cycle, so "did the menu actually reach the window today" is a question with an answer rather than an assumption. Pagination stops at a page cap and says so - a truncated chain is loud, not silent.

A second opinion from a prediction market

Kalshi's daily index-close markets are a crowd-implied distribution of today's close - a second, independent estimate to weigh against what the option chain implies. Handed to the model as a *prior, not a signal*: no code acts on it, so it can only persuade.

```
SPY  prior close 7,712, median 7,688 (−0.31%); P(above) 0.293; volume 756  -> shown
QQQ  prior close 29,433, median 29,450 (+0.06%); P(above) 0.514; volume 45  -> withheld
```

The harder half is knowing when to ignore it. One run, both outcomes. A barely-traded market still quotes every bucket just as confidently, and nothing about the QQQ numbers looks broken - that is the point. Two gates decide: minimum volume, and entropy over log(buckets). Nobody had put money behind QQQ, so the model was told nothing rather than something unearned.

Withheld priors are journalled *with the reason*, so "no second opinion today" is answerable rather than absent. Public API, no key, never traded.

...and then we grade it

A prior nobody scores is decoration. Every night `eod_review.py` Brier-scores the probabilities the model was actually handed against what the market did - `(p − outcome)²`, where a coin flip scores **0.25**.

Tue Sep 1, judged account	Brier	
Kalshi index-close market	**0.004**	confident, and right
The option chain's own odds	**0.008**	agreed, slightly less sharply
Priors the gate *withheld*	0.006	shadow-graded: the gate was not discarding good information

One day and a market with a clear direction - not a claim about edge. The point is that the question is answerable at all: we grade the inputs, not just the model, so "was the second opinion worth listening to" stops being a matter of taste. Separately, the model's own citations of those numbers are checked against the prior it was given.

The agent investigates before it acts

```
research: get_bars({'symbol': 'SPY', 'timeframe': '15Min'})        -> 3334 chars
research: get_stock_snapshot({'symbol': 'NVDA'})                   -> 622 chars
research: get_option_snapshot({'symbols': 'NVDA260902P00220000,...'}) -> 1339 chars
research: get_news({'symbols': 'NVDA', 'limit': 5})                -> 1770 chars
model output: [{"instrument":"option","symbol":"NVDA260902P00220000","side":"buy",
  "qty":10,"order_type":"limit","limit_price":4.65,
  "reason":"NVDA is in a clear intraday downtrend (225 -> 218.6); the 220 put with
           4 DTE, delta -0.58, aligns with the bearish momentum"}]
```

Four read-only tools mapped to Alpaca MCP calls, six calls max, every one journaled. Nothing that orders is reachable.

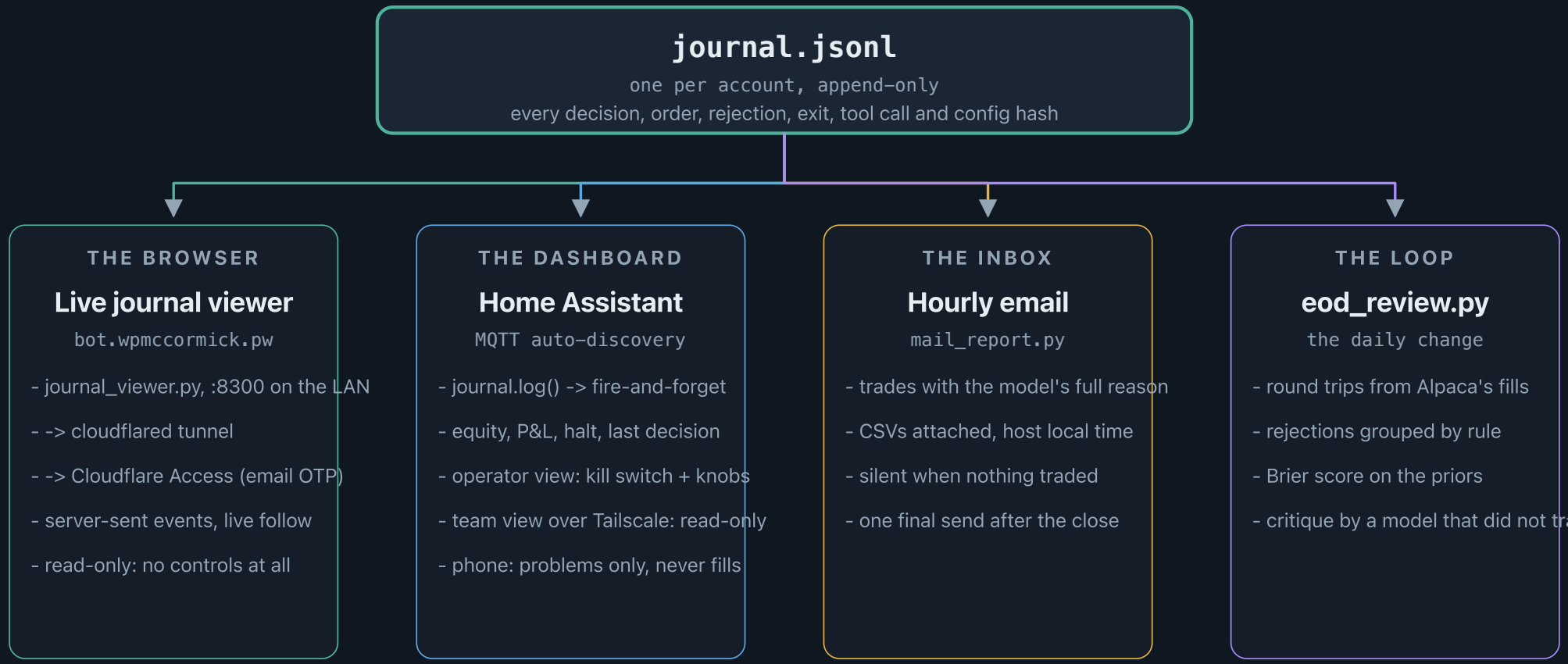
The leash

Per proposal	whitelist - ≤ $5,000 per position - ≤ 4 positions - ≤ 10 contracts/order - 2-45 DTE on entries - entries 09:45-15:15 ET
Per cycle, before the model	close on expiry day - stop-loss -40% - take-profit +60%
Resting orders	an unfilled buy counts as held, and the next cycle cancels it - the caps bound *committed* exposure, not just what filled
Per day	2% loss → flatten everything and halt - `HALT` file = global kill switch
End of day	close contracts expiring within a day; hold the rest under the stops (judged on Thursday's close)

```
REJECTED buy 12 SPY260904C00775000: qty 12 exceeds max_contracts_per_order 10
```

Stocks and options are both first-class. Every limit above applies to either one - the dollar cap counts `contracts x 100 x price` or share market value, and the position count is combined. The model states which instrument it wants and why: options when the edge is time-bounded, shares when volatility is expensive or the move is a slow grind.

Everything is journaled - and four things read it



One file, four windows. Three are read-only views for humans; the fourth decides what changes tomorrow.

Watching it think, from anywhere

The journal, streamed to a browser over server-sent events. LAN-bound, published by a Cloudflare tunnel behind Access with an email one-time PIN - no VPN, no account, nothing to install: **bot.wpmccormick.pw**. Live follow, per-account filters, replay by date.

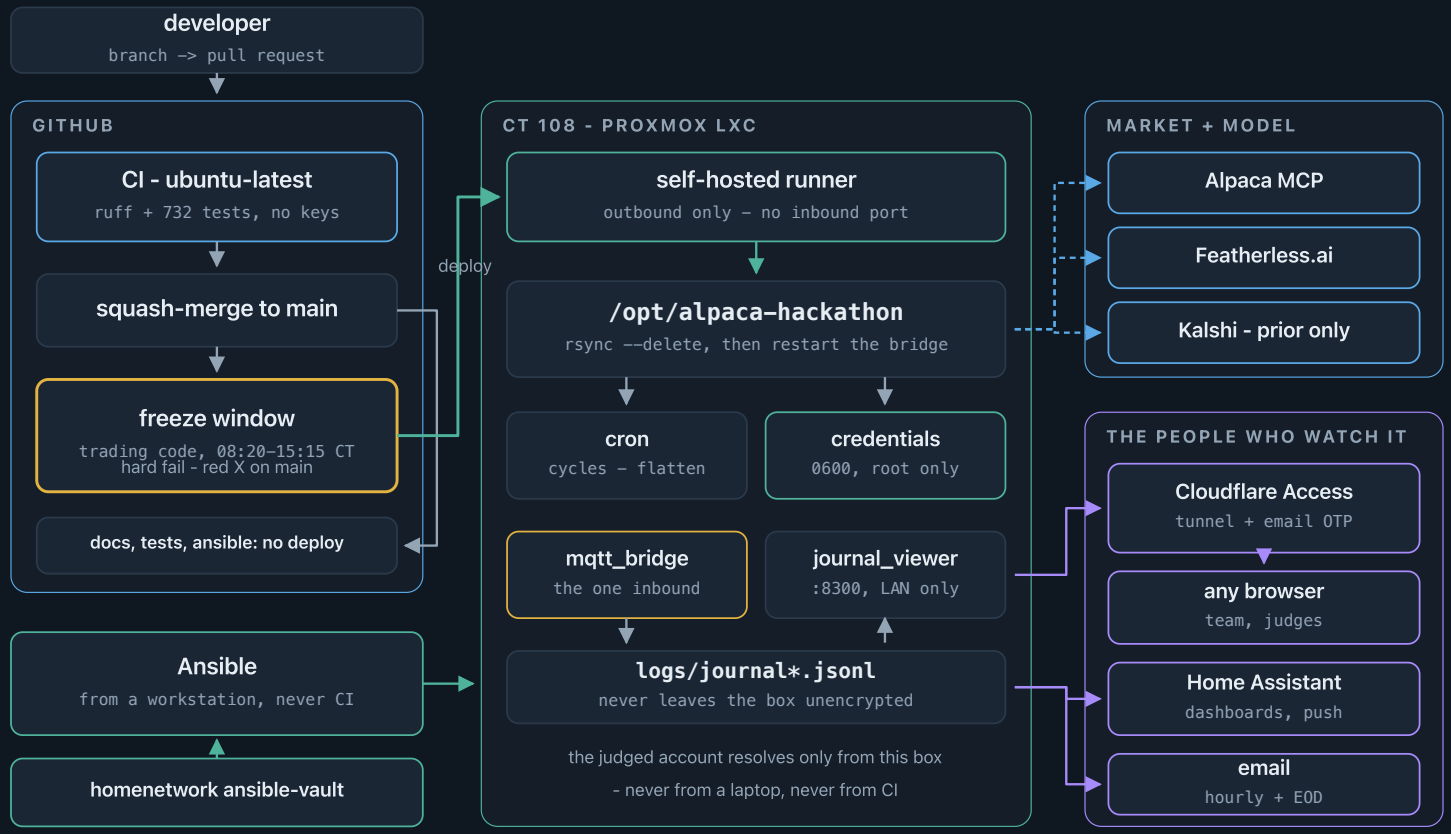
It found four bugs in two days. None was a failing test - in each the code did exactly what it had been told:

#174	the feed cut the model's reasoning mid-word at 220 characters
#170	closing a +$1,340 winner, the model named a *neighbouring strike*, three cycles running
#171	an unfilled limit sat invisible; ten minutes on, the same thesis was bought again next door
#173	a ten-minute-old position proposed for exit on a "weakening thesis" - every number it cited had moved *in its favour*

All four fixed and deployed. A test asserts what you already thought of; reading the reasoning live is how you find the rest.

It runs on our own hardware, not a laptop

A squash-merge is live in about a minute - **113 deploys this week**.



A merge deploys in about a minute. Nothing reaches the container inbound except the tunnel and the broker.

Built to be changed while it is running

A four-day agent is mostly an *iteration speed* problem: the constraint is not writing the strategy, it is changing it without breaking a live account.

732 tests, 2s	every one runs with **no API keys and no network** - clone and get a green suite before you have credentials
Only Docker needed	`docker compose run --rm bot` - tests, a full dry-run cycle, the docs site
Rehearse any time	`--dry-run` prints orders instead of sending; `--force` skips the market gates - a whole cycle is testable at 2am Sunday
Strategy changes without a deploy	`override.py` - allowlisted knobs, hard risk caps stay git-only, all of it expires at the close

Dozens of those tests exist because something broke - a report that published on one of four cycle paths, a test that passed only on a laptop. Each incident ends as a test, so the suite is a list of things that can no longer happen quietly.

The daily loop

16:05 `eod_review` → read: round trips, exit mix, rejections by rule, a second model's read →

one change → `override.py` (expires at close) or config PR →

CI → self-hosted runner deploys before the open

Two more paper accounts run variant configs against the same live market - the only backtest available in four days. 132 PRs, 732 tests, 113 automatic deploys - in seven days.

The experiment farm - real A/B, not a backtest

Four days isn't enough for a backtest, and the free feed carries no historical options data anyway. So the A/B runs *live*: three paper accounts on the same box, same ten-minute cadence, same market, different configs - a real experiment rather than a replay.

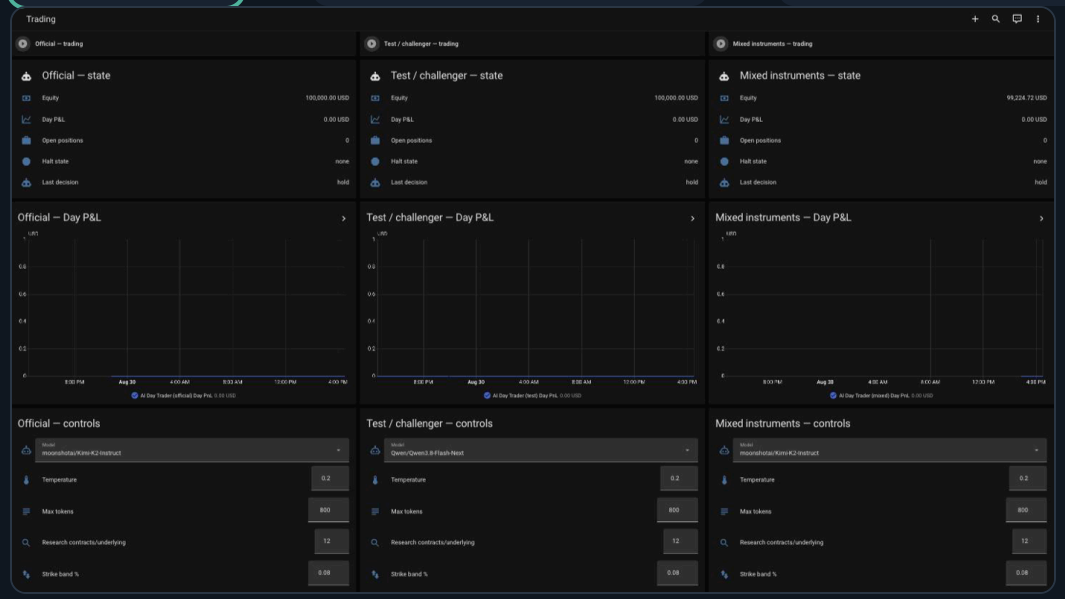
```
# on the trading host, right now — every 10 min, 08:00–14:50 CT
run_cycle.py --account official
run_cycle.py --account test  --config config-test.yaml
run_cycle.py --account mixed --config config-variants/mixed.yaml
```

Each is one deliberate change. `test` swaps the model and enables research tools and learning; `mixed` differs by exactly *one key* - stock and options as peers, not options-first - so "which instrument should this agent reach for" is answered by P&L rather than by us.

The same thing runs on a laptop. `docker compose` brings up the bot, the variant farm and the docs site self-contained - no host, no Proxmox, none of our keys. It is how the project is developed, and it would serve just as well to run it.

Home Assistant, over MQTT - fully decoupled

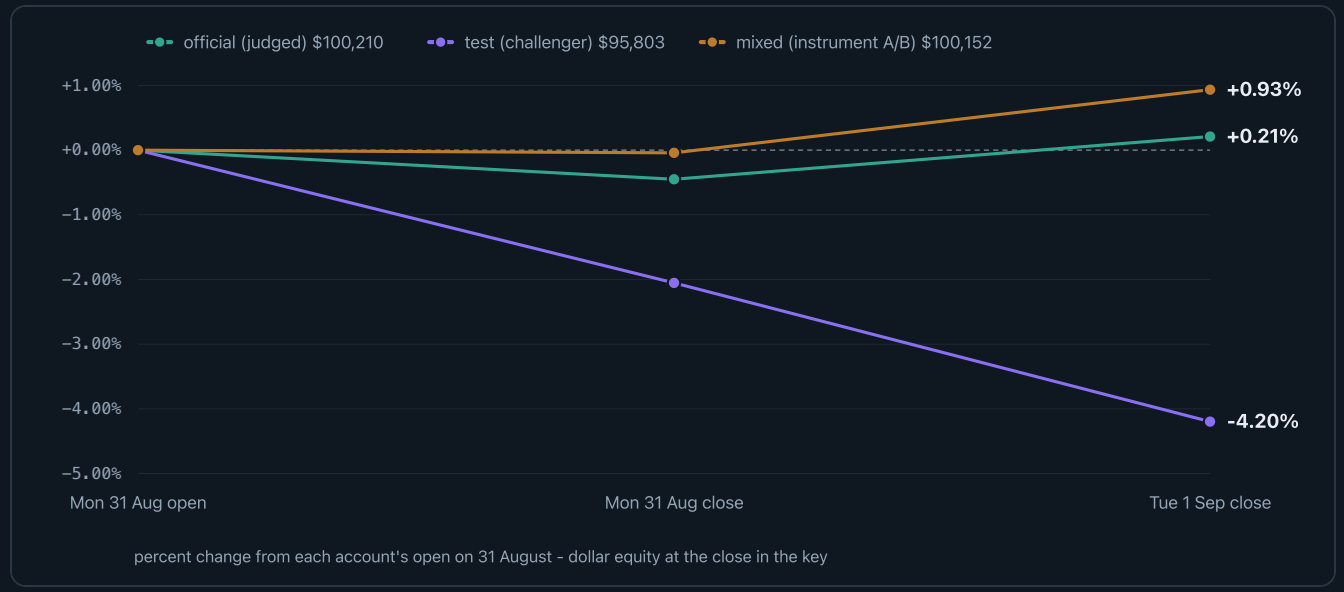
journal.log() → MQTT (fire-and-forget) → HA auto-discovery



One feed, three audiences. **Operator:** kill switch and knobs. **Team:** a read-only copy over Tailscale - HA has no per-entity permissions, so the separation has to *be* the dashboard. **Phone:** problems only.

A fill sends *no* alert, on purpose: a channel that buzzes on routine trades gets muted, and the halt alert with it. What buzzes is *silence* - no cycle for 25 minutes, the one failure a dashboard cannot show.

Results (fill Thu Sep 3 after the close)



Equity Mon open → Thu close: $___ → $___ (__%)

Round trips: __ - exits: __ stop / __ take-profit / __ expiry / __ model

Official vs challenger vs mixed: __

What didn't work: __

What's next, and thanks

- Promote what the challenger proved: research tools, prediction-market prior, learning loop
- Multi-leg spreads once the gates cover assignment risk
- Home Assistant over MQTT: the light that goes red when the leash pulls

github.com/bill-mccormick-dg/alpaca-hackathon - MIT

Thanks: Alpaca, lablab.ai, Featherless AI